SnapVault — Data Model

Entity reference for the JPA/Hibernate entity layer — v3, UUID primary keys + audit columns

SnapVault is a private client-gallery delivery platform. A photographer creates galleries, uploads photos into them, and invites one or more clients to view and favorite the photos in a given gallery. This document defines the five core entities, their fields, and the relationships between them, as the reference for hand-written @Entity classes in the entity package. All primary keys use java.util.UUID with GenerationType.UUID, applied uniformly across every entity.

User

Represents both photographers and clients — distinguished by role, not by separate tables.

Field	Type	Notes
id	UUID	@Id, @GeneratedValue(strategy = GenerationType.UUID)
email	String	unique, not null
passwordHash	String	not null
role	Role (enum: PHOTOGRAPHER, CLIENT)	@Enumerated(EnumType.STRING)
lastLoginAt	LocalDateTime	nullable — most recent login, distinct from createdAt
createdAt	LocalDateTime	@CreationTimestamp, not null, not updatable
updatedAt	LocalDateTime	@UpdateTimestamp, not null

Gallery

The core unit of work — one shoot, owned by one photographer.

Field	Type	Notes
id	UUID	@Id, @GeneratedValue(strategy = GenerationType.UUID)
title	String	not null
shootDate	LocalDate	
photographer	User	@ManyToOne, FK photographer_id (UUID), not null
createdAt	LocalDateTime	@CreationTimestamp, not null, not updatable
updatedAt	LocalDateTime	@UpdateTimestamp, not null — tracks title/date edits

Invitation

Links a Gallery to an invited person. A gallery can have many invitations (multiple relatives); a user can appear on many invitations across different galleries over time.

Field	Type	Notes
id	UUID	@Id, @GeneratedValue(strategy = GenerationType.UUID)
gallery	Gallery	@ManyToOne, FK gallery_id (UUID), not null
clientEmail	String	not null — email the invite was sent to
user	User	@ManyToOne, FK user_id (UUID), NULLABLE until the invitee registers/logs in
status	InvitationStatus (enum: PENDING, ACCEPTED)	@Enumerated(EnumType.STRING)
createdAt	LocalDateTime	@CreationTimestamp, not null, not updatable
updatedAt	LocalDateTime	@UpdateTimestamp, not null — tracks the PENDING→ACCEPTED flip

Photo

An individual photo belonging to one gallery.

Field	Type	Notes
id	UUID	@Id, @GeneratedValue(strategy = GenerationType.UUID)
gallery	Gallery	@ManyToOne, FK gallery_id (UUID), not null
filePath	String	not null
thumbnailPath	String	nullable until thumbnail generation runs
uploadedAt	LocalDateTime	@CreationTimestamp, not null — doubles as this entity's createdAt
updatedAt	LocalDateTime	@UpdateTimestamp, not null — captures when thumbnailPath is set by async generation

Favorite

Join between a client and a photo they've liked.

Field	Type	Notes
id	UUID	@Id, @GeneratedValue(strategy = GenerationType.UUID)
user	User	@ManyToOne, FK user_id (UUID), not null
photo	Photo	@ManyToOne, FK photo_id (UUID), not null
createdAt	LocalDateTime	@CreationTimestamp, not null
—	—	Consider a unique constraint on (user_id, photo_id)

Relationships & cardinality

- User (photographer) 1 → * Gallery
- Gallery 1 → * Photo
- Gallery 1 → * Invitation
- Invitation * → 1 User (nullable, the invited person)
- User (client) 1 → * Favorite ← * Photo

Note: every relationship above is modeled as @ManyToOne on the "many" side. Bidirectional @OneToMany accessors (e.g. Gallery.getPhotos()) are optional conveniences that can be added later once the base model compiles and boots correctly — they are not required for correctness.

Design notes

Entities are plain mutable classes, not Java records — Hibernate requires a no-arg constructor and post-construction field mutation via reflection/proxies, which conflicts with record immutability. Records remain the right choice for DTOs in the API layer, not for the entity layer.

Primary keys use java.util.UUID (not String) with GenerationType.UUID, applied uniformly across all five entities — a direct JPA 3.1 / Hibernate 6 strategy, no @GenericGenerator boilerplate needed. The Java field type is UUID rather than String for compile-time type safety; Jackson serializes it as a string automatically, and Spring converts @PathVariable UUID from the URL path with no extra code. Random (v4) UUIDs trade some B-tree index locality for global uniqueness with no coordination across services — an acceptable tradeoff at this scale, and worth naming as a conscious choice rather than a default.

Audit columns: createdAt and updatedAt are set via Hibernate's @CreationTimestamp / @UpdateTimestamp annotations, never assigned manually in service code, so they can't go stale. updatedAt is added wherever a row is mutated after creation: User, Gallery, Invitation, and Photo (whose thumbnailPath is filled in later by async thumbnail generation, so updatedAt records when that happened). Favorite is the only immutable entity — a favorite is added or removed, never edited — so it carries only createdAt. Photo reuses its existing uploadedAt field as its creation timestamp rather than adding a redundant createdAt.